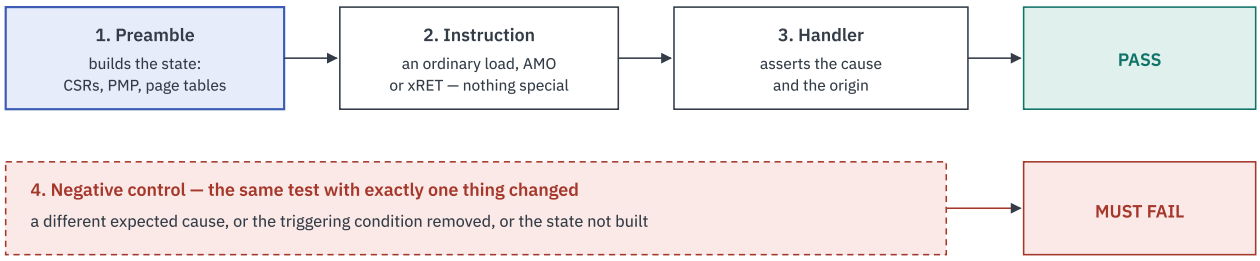
RISC-V SAIL TEST GENERATION · PRIVILEGED SCENARIOS

PMP, Traps, PMA and Virtual Memory

Four areas that cannot be reached by choosing an opcode. Each is an ordinary instruction executed in a state the preamble constructs — and each ships with a control that must fail.

The shape they all share



If a control ever passes, the positive result it guards is worthless — so the control is part of the test, not commentary.

The instruction is never the interesting part. The state before it is.

> **Why controls are mandatory here specifically.** Two of these scenarios passed *vacuously* during development and were caught only by constructing the case that had to fail. A privileged test that never reaches its trap looks exactly like one that traps correctly — both end with a pass.

---

1 · PMP

Why an opcode cannot reach it

PMP is not an instruction. It is a check the model runs on *every* memory access, against entries held in `pmpcfg` / `pmpaddr` CSRs. To exercise it you must program those entries, then make an access that lands inside or outside them.

Foundation it needs

REQUIREMENT	WHERE
Struct-typed vector elements	PMP config is `vector(64, Pmpcfg_ent)` — a vector of bitfield *structs*. Without the engine fix, any symbolic PMP access crashes. **This is the single blocker**
Concrete `reset_pmp()`	Model-side: builds a concrete entry per register rather than updating a symbolic prior value
Preamble flags	`--pmp-allow-all` installs a catch-all entry; `--pmp-deny <addr>` installs a denying entry 0 plus a permitting entry 1
Privilege control	`--run-in-supervisor`, and `--preload-xpp mpp=u` with `--preload-xepc` for U-mode

The scenarios

SCENARIO	FLAGS	EXPECT
load permitted (M)	—	pass
load denied (M)	`--pmp-deny 0x80020000 --expect-trap-cause 5 --trap-is-pass`	pass
NEG wrong cause	`--expect-trap-cause 7`	fail
NEG no violation	`--pmp-deny 0x80028000` (a different address)	fail
permitted / denied (S)	`--run-in-supervisor …`	pass
NEG S default-deny	`--run-in-supervisor` with no allow-all	fail
permitted / denied (U)	`--preload-xpp mpp=u …`	pass

> **Why the same access is repeated at M, S and U.** M-mode is the *least* discriminating privilege for PMP: an access matching no entry is *allowed* in M and *denied* in S and U. Testing only M therefore exercises the permissive arm of `pmpCheck` and leaves the default-deny path — the one that actually protects anything — unexecuted.

> **A gap we state rather than hide.** `pmpCheck`'s final line — `if priv == Machine then None() else Some(accessFault…)` — is the *unmatched* case, the only place privilege alone decides the outcome. No scenario reaches it, because both PMP options guarantee a match: allow-all installs a catch-all, and deny installs a denying entry *plus* a permitting one. Leaving the data address unmatched while keeping the harness's own code fetchable needs a bounded-range option the generator does not have yet (`--pmp-allow-range lo-hi`). Until then the arm is covered only by a control that fails by design, and so contributes no coverage.

---

2 · Traps, exceptions and interrupts

Why an opcode cannot reach it

An exception needs a faulting condition constructed; an interrupt is asynchronous and a self-checking ELF cannot wait for a timer across four simulators with four notions of time.

Foundation it needs

A trap handler emitted by the generator, plus preamble flags that make delivery deterministic. **The interrupt is not awaited — its architectural conditions are set directly.**

FLAG	WHAT IT SETS	WHAT IT TESTS
`--pending-interrupt`	Pending bit and `mie`, but **not** `mstatus.MIE`	`WFI`'s wake condition, deliberately *not* delivery
`--deliver-interrupt`	Adds the global enable, so the trap is actually taken	Delivery
`--delegate-interrupt`	`mideleg`, so it lands in S-mode; handler reads `scause`	Delegation
`--preload-xepc / --preload-xpp`	`mepc` / `sepc` and `MPP` / `SPP`	Trap return and privilege transitions

The scenarios

SCENARIO	DETAIL	EXPECT
ecall / ebreak	causes 11 and 3	pass
MRET to M / S / U	each with the PMP entry the new privilege needs	pass
SRET to S / U	as above	pass
NEG MRET to U, no PMP	the post-return fetch must fault, because the privilege change really happened	fail
interrupt delivered	cause 3, taken for real	pass
interrupt delegated	**cause 1**, in S-mode	pass
NEG wrong cause	right delivery, cause 7	fail

> **Delegation must use a *supervisor* software interrupt — cause 1, not 3.** `legalize_mideleg` hardwires the machine-level delegation bits to zero, so a machine software interrupt cannot be delegated at all. The first version tried cause 3 and read as a delegation failure when it was an impossible test.

> **An interrupt cause is the cause code with the top bit set**, so the handler tests that bit *as a sign bit* rather than materialising `0x8000_0000_0000_0003` — which keeps it XLEN-independent. Requiring the bit to be set also stops an exception with the same low cause number satisfying an interrupt test by accident: cause 3 as an exception is `ebreak`.

> **A control we removed because it tested nothing.** An `ebreak` control was tried alongside the delivered interrupt. With delivery enabled the interrupt fires *before* the `ebreak` retires, so the test passed on the interrupt and the `ebreak` was never reached. Removed rather than left in place looking like coverage.

---

3 · PMA — the one that works differently

Why an opcode cannot reach it, and neither can a preamble

Physical memory attributes are **platform-defined region properties**, not architectural state any instruction can write. There is no CSR to program. So the state is not built by a preamble at all — it is built by **generating a configuration**.

How a PMA test is made

```
1. take the Golden Model's own config JSON
2. flip one region attribute      e.g. atomic_support: "AMONone"
3. run an ordinary access there   e.g. amoadd.d x1, x2, (x3)
4. expect the resulting fault     cause 7, store/AMO access fault
```

SCENARIO	ATTRIBUTE CHANGED	ACCESS	EXPECT
AMO unsupported	`atomic_support: AMONone`	`amoadd.d`	cause 7

SCENARIO	ATTRIBUTE CHANGED	ACCESS	EXPECT
NEG wrong cause	`same config`	`amoadd.d`	cause 5 → fail
LR unreservable	`reservability: RsrvNone`	`lr.d`	cause 5
NEG wrong cause	`same config`	`lr.d`	cause 7 → fail

> **Two attributes, two different fault classes.** Deliberate: a single over-broad configuration change cannot satisfy both, so the pair proves the specific attribute was what mattered rather than the config being generally broken.

> **The obvious control does not work here, and the reason is instructive.** The first attempt was "the same ELF under the stock config" — which sounds exact. It does not discriminate: with no trap, the harness falls through to the ordinary final-state comparison, which the generator solved for the *non-trapping* path, so it passes either way. The control that does discriminate is the same config with the **wrong expected cause**, which proves the handler compares the cause rather than merely noticing a trap.

---

4 · Virtual memory and translation

Why an opcode cannot reach it

A page-table walk needs a page table in memory, `satp` pointing at it, and an access to an address that is or is not mapped. Three constructed things before the instruction runs.

Foundation it needs

REQUIREMENT	DETAIL
A page table builder	Behind `--sv39`: writes a real table, identity-maps the code region, points `satp` at it
S-mode	`--run-in-supervisor` — translation is not applied in M-mode
PMP first	`--pmp-allow-all`, or the access faults on PMP before translation is ever consulted
Two addresses	One inside the identity map, one outside it, as separate opcode sequences

The scenarios

SCENARIO	WHAT IT PROVES	EXPECT
supervisor, Bare	S-mode works with translation off — the baseline	pass
Sv39, mapped address	A real walk resolves and the access succeeds	pass
Sv39, unmapped address	The walk fails and raises a **page** fault, cause 13	pass
NEG no translation	Same unmapped access with Sv39 off — faults *differently*	fail
NEG mapped address	Sv39 on, mapped address — does not fault at all	fail

Cause 13 is the whole point. A page fault is reachable only through a real walk — no other mechanism produces it. The two controls bracket it from both sides: translation off gives a different fault, a mapped address gives none. Together they prove the walk happened, which a single passing test cannot.

A live limitation. The mapped-address sequence uses `slli x2, x2, 32` to build the address, which is RV64-only — so every one of these scenarios currently fails at RV32. It needs either an RV32 address sequence or an explicit skip. Reported as a framework fault, not a model one.

What is common, and what it costs

AREA	STATE BUILT BY	ASSERTION	CONTROL VARIES
PMP	CSR entries in the preamble	access fault, cause 5 or 7	the cause, and the denied address
Traps	CSR preloads in the preamble	cause, and the mode it landed in	the cause, and the PMP entry
PMA	**a generated configuration**	fault class per attribute	the expected cause
Virtual memory	a page table plus `satp`	page fault, cause 13	translation on/off, address mapped/not

Three of the four build state in the preamble; PMA builds a configuration. That is the exception worth remembering, and it is why PMA needed no new generator capability at all — only a config-mutation step and somewhere to put the variant.

Scenarios, flags and expectations read from `python-isla/scenario_tests.py`; the gaps are recorded in that file's own comments rather than discovered here.